

Verantwoordingsdocument

Eindopdracht Backend



NOVI
HOGESCHOOL



Rick van Campen

NOVI Hogeschool

Inleverdatum: 11 juli

Inhoudsopgave

Technische keuzes	2
• <i>Gebruik van Spring Boot Framework.....</i>	<i>2</i>
• <i>Gebruik van Spring Security met JWT voor authenticatie.....</i>	<i>2</i>
• <i>Gebruik van Lombok</i>	<i>2</i>
• <i>Gebruik van PostgreSQL als database</i>	<i>2</i>
• <i>Gebruik van DTO's.....</i>	<i>2</i>
• <i>Gebruik van goede Service-Repository structuur.....</i>	<i>3</i>
• <i>Gebruik van Java 11 als JDK</i>	<i>3</i>
• <i>Gebruik van Git en GitHub</i>	<i>3</i>
• <i>Gebruik van Hibernate.....</i>	<i>3</i>
• <i>Gebruik van global exception handling</i>	<i>3</i>
Beschrijving limitaties	4
• <i>Taalconsistentie.....</i>	<i>4</i>
• <i>Commentaar in de code</i>	<i>4</i>
• <i>Geen gebruik van pull requests en branches in GitHub</i>	<i>4</i>
Link naar GitHub.....	5

Technische keuzes

- **Gebruik van Spring Boot Framework**

Ik heb gebruik gemaakt van Spring Boot Framework, omdat dit een sterk raamwerk is voor het ontwikkelen van webapplicaties met Java. Zo biedt Spring Boot Framework uitgebreide ondersteuning aan een aantal zaken zoals: beveiliging (Spring Security), dependency injection en integratie met databases via Spring Data.

- **Gebruik van Spring Security met JWT voor authenticatie**

Ik heb gebruikt gemaakt van Spring Security en dan met name de JWT (JSON Web Tokens) voor de authenticatie. JWT biedt een stateless authenticatiemechanisme, simpelweg dat het systeem geen interne toestand of sessie bijhoudt voor elke gebruikersverbinding, wat erg handig is voor RESTful API's waarbij dus elke client-aanvraag onafhankelijk is. Spring Security biedt dus een goede ondersteuning voor authenticatie, maar ook autorisatie en JWT's bieden een efficiënte manier om authenticatietokens te behoren. Zo krijg je bij het inloggen een bearer token terug.

- **Gebruik van Lombok**

Ik heb gebruik gemaakt van Lombok, want het helpt bij het verminderen van herhalende code, omdat Lombok simpelweg automatisch standaardmethoden genereert, zoals: getters, setters, constructors en nog andere standaardmethoden. Dit zorgt er uiteindelijk voor dat de code makkelijker te lezen is en de code is makkelijker te onderhouden.

- **Gebruik van PostgreSQL als database**

Ik heb gebruik gemaakt van PostgreSQL, omdat dit een sterke, open-source relationele database is die tevens goed samenwerkt met Spring Boot. Zo is de database in staat om goede transactieondersteuning te bieden, maar het levert ook betrouwbare prestaties en is goed genoeg voor productietoepassingen. Daarnaast is het relatief makkelijk om naar een andere database te switchen, omdat de applicatie en database onafhankelijk van elkaar zijn. Dit biedt flexibiliteit voor toekomstige aanpassingen in de database-infrastructuur.

- **Gebruik van DTO's**

Ik heb gebruik gemaakt van DTO's voor data-overdracht. De DTO's helpen me bij het scheiden van interne entiteiten van de applicatie van de data die over de netwerkinterfaces wordt verzonden. Ze bieden dus een gestandaardiseerde manier om gegevens te valideren en onbedoelde overdracht van gevoelige gegevens te voorkomen.

- **Gebruik van goede Service-Repository structuur**

Ik heb gebruik gemaakt van een goede Service-Repository structuur, omdat dit de scheiding van de verantwoordelijkheden (SoC) bevordert door logica (Services) te scheiden van gegevensbeheer (Repositories). Dit zal uiteindelijk een aantal zaken makkelijker maken, zoals: unit testing, onderverdeling van modules en de herbruikbaarheid van de code.

- **Gebruik van Java 11 als JDK**

Ik heb gebruik gemaakt van Java 11, omdat Java 11 een Long-Term Support versie is met verbeterde prestaties en beveiligingsupdates. Ook biedt Java 11 functies, zoals lokale variabele type inferentie (var) en HTTP Client API. Dezen zijn erg handig bij moderne Java-ontwikkeling.

- **Gebruik van Git en GitHub**

Ik heb gebruik gemaakt van Git en GitHub voor versiebeheer. Git ondersteunt samenwerking en versiebeheer door het bijhouden van wijzigingen in de code. GitHub daarentegen biedt een platform voor het delen van code, het beheren van projecten en het reviewen van wijzigingen. Erg handig dus voor tijdens het bouwen van een applicatie.

- **Gebruik van Hibernate**

Ik heb gebruik gemaakt van Hibernate als ORM-framework, omdat ik hierdoor gegevens uit mijn relationele database makkelijk en efficiënt kan opslaan en ophalen. Handmatig geschreven SQL-code wordt hierdoor geminimaliseerd en gegevensbeheer biedt een objectgeoriënteerde benadering.

- **Gebruik van global exception handling**

Ik heb gebruik gemaakt van global exception handling mechanisme. Hierdoor kunnen fouten en uitzonderingen op een gestandaardiseerde manier worden afgehandeld in mijn applicatie. Dit verbetert de weerbaarheid van mijn applicatie door ervoor te zorgen dat onverwachte situaties op een gecontroleerde manier worden behandeld. Ook biedt het de mogelijkheid om aangepaste foutmeldingen terug te sturen naar de gebruiker. Dit zal uiteindelijk de gebruikerservaring verbeteren en het debuggen zal nu makkelijker gaan.

Beschrijving limitaties

- **Taalconsistentie**

Tijdens het ontwikkelen van mijn applicatie heb ik Nederlands en Engels door elkaar gehaald, bijvoorbeeld met de namen van de Java-bestanden. Dit kan dus leiden tot verwarring wanneer beide talen niet worden beheerst. De volgende keer is het voor mij noodzakelijk om alles in het Engels te doen, zodat alle ontwikkelaars mijn applicatie zullen begrijpen.

- **Commentaar in de code**

Tijdens het ontwikkelen van mijn applicatie heb ik eigenlijk geen commentaar toegevoegd in de code. Ik heb hier tijdens het ontwikkelen niet aan gedacht. Een aantal punten in mijn code zouden eigenlijk wel enige vorm van uitleg nodig hebben. Daarom is het noodzakelijk dat ik de volgende keer goed oplet welke punten in mijn code commentaar nodig hebben en dat ik dit er dan ook bijzet in de code.

- **Geen gebruik van pull requests en branches in GitHub**

Tijdens het project heb ik eigenlijk geen gebruik gemaakt van pull requests en branches. Dit zou eigenlijk wel moeten om het versiebeheer beter te waarborgen van de applicatie. De volgende keer moet ik er dus voor zorgen dat ik wel gebruik maak pull requests en branches. Hierdoor wordt het versiebeheer beter gewaarborgd en kan eventuele samenwerking met andere ontwikkelaars beter verlopen.

Link naar GitHub

https://github.com/RickvanCampen/Eindopdracht_Backend